

Relatório APF-T — Sistema de Gestão de Coworking

Autores: Bruno Pereira (Nº 112726) · Rafael Claro (Nº 088860) **Disciplina:** Base de Dados — LECI **Data:** 2026-05

1. Introdução

Este projeto implementa o sistema de gestão de um operador de coworking com vários espaços físicos, salas de reunião reserváveis à hora e postos de trabalho atribuídos por adesão ou alugados ao dia. Cobre o ciclo de vida completo de cliente, adesão, reserva e pagamento, e expõe à aplicação C# Windows Forms um conjunto de stored procedures, views e funções que encapsulam toda a regra de negócio.

A entrega APF-E definiu requisitos, DER e esquema relacional. A APF-T concretiza o modelo em SQL Server 2019, com schema, índices, 16 triggers, 11 views de consulta, 7 UDFs e SPs de negócio, auditoria via system-versioned tables, e plano de testes documentado.

2. Arquitetura



A aplicação invoca a BD exclusivamente via **SqlCommand** parametrizado (sem concatenação de strings) — toda a lógica de negócio fica encapsulada em SPs/triggers e a app é uma fina

camada de apresentação. Vantagens:

- Elimina vetores de SQL injection.
- Centraliza a regra de negócio na BD (a app não pode contornar triggers/validações).
- A camada de apresentação pode ser reescrita (web, mobile) sem tocar nas regras.

3. Modelo de Dados

3.1 Tabelas principais

Tabela	PK	Resumo
cliente	cliente_id	Identidade fiscal (NIF), email único
plano	plano_id	Catálogo de planos Flex/Fixo/Privado
espaco	espaco_id	Edifício/sala física com horário
recurso	recurso_id	Supertype — abstrai sala vs. posto
sala	recurso_id (FK → recurso)	Subtype com capacidade, preço/hora
posto	recurso_id (FK → recurso)	Subtype com tipo (Flex/Fixo/Privado), preço/dia
adesao	adesao_id	Cliente ↔ Plano com preço snapshot + recurso opcional
reserva	reserva_id	Cliente ↔ Recurso para data/horas
pagamento	pagamento_id	Liga-se a exactly one adesão XOR reserva
politica_cancelamento	politica_id	Tiers de reembolso por antecedência
notificacao	notificacao_id	Eventos enviados ao cliente
lista_espera	lista_espera_id	Cliente ↔ Recurso quando indisponível

3.2 Decisões de modelo

Recurso supertype. Sala e posto partilham operações (reservar, ver disponibilidade) mas têm atributos próprios. Em vez de tabela única com colunas nulas, optei pelo supertype `recurso` + subtypes `sala` / `posto` ligados via PK==FK com `ON DELETE CASCADE`. Vantagens: a FK em `reserva.recurso_id` aponta para o supertype, simplificando o trigger T1 de sobreposição. Custo: dois JOINS para obter os detalhes (atenuado por índices).

Posto via adesão. O atributo `adesao.recurso_id` (NULL para Flex, NOT NULL para Fixo/Privado) materializa a regra "Fixo/Privado têm posto atribuído". Trigger T9 enforça a coerência `tipo_plano ↔ tipo_posto`. Alternativa rejeitada: tabela ponte `adesao_posto` — overhead sem ganho expressivo.

Pagamento como XOR + snapshot de preço. A constraint `ck_pagamento_servico` garante que `adesao_id` e `reserva_id` são mutuamente exclusivos e que pelo menos um está preenchido. Em resposta à pergunta do professor "qual o preço?", o `pagamento` carrega ainda uma coluna `preco_servico_snapshot DECIMAL(10,2) NOT NULL`, que fotografa o preço do serviço subjacente (`reserva.valor` ou `adesao.preco_acordado`) no momento da criação. A ligação ao valor cobrado deixa de viver apenas no trigger e passa a ser explícita no schema:

- `ck_pagamento_valor_snapshot CHECK (valor = preco_servico_snapshot)` — o que se paga tem de ser o que se cobrou.
- Trigger T6 — `preco_servico_snapshot` tem de coincidir com o preço actual do serviço no momento da inserção.
- Trigger T8 — `cliente_id` tem de ser o titular do serviço.

Esta redundância controlada protege o histórico contra futuras alterações de preço (alinhada com `adesao.preco_acordado`) e dá ao auditor uma única coluna para responder à pergunta acima.

4. Normalização

Todas as tabelas estão em **BCNF**:

Tabela	Dependências funcionais não-triviais	Análise
<code>cliente</code>	<code>cliente_id → nome, nif, email, ... ; nif → cliente_id ; email → cliente_id</code>	3NF: todas saem da chave. BCNF: nif e email são candidate keys (UNIQUE), por isso à esquerda da DF — OK.
<code>plano</code>	<code>plano_id → nome_plano, tipo_plano, preco_mensal, ... ; nome_plano → plano_id</code>	BCNF.
<code>espaco</code>	<code>espaco_id → nome, morada, ... ; nome → espaco_id</code>	BCNF.
<code>sala</code>	<code>recurso_id → espaco_id, nome, capacidade, ... ; (espaco_id, nome) → recurso_id</code>	BCNF.

posto	análogo a sala	BCNF.
adesao	adesao_id → cliente_id, plano_id, ...	3NF/BCNF. Notar: preco_acordado é snapshot — duplica plano.preco_mensal no momento do INSERT mas justifica-se porque o preço pode mudar e a adesão tem de preservar o valor histórico (data temporal). Esta "redundância controlada" é decisão consciente.
reserva	reserva_id → cliente_id, recurso_id, data, horas, ...	BCNF.
pagamento	pagamento_id → cliente_id, valor, preco_servico_snapshot, ...	BCNF. preco_servico_snapshot é redundância controlada (snapshot do preço do serviço no momento do pagamento — paralelo a adesao.preco_acordado).

Não existe violação 3 → BCNF porque as únicas DFs não-triviais saem ou de PKs ou de UNIQUE keys (candidate keys), pelo que toda DF tem lado esquerdo superkey.

5. Robustez aplicacional

A app comunica com a BD exclusivamente via `SqlCommand` parametrizado (`cmd.Parameters.AddWithValue(...)`). Não há concatenação de strings SQL, não há `EXEC(@sql)` na BD. Todas as operações DML passam por SPs com `SET XACT_ABORT ON` que fazem rollback automático em qualquer erro intra-transacção.

Resultado: SQL injection é inexistente por construção; a regra de negócio fica encapsulada em SPs/triggers; a camada de apresentação é fina (UI + chamadas a SPs) e pode ser substituída sem tocar nas regras.

6. Concorrência

O trigger T1 (`trg_reserva_sem_sobreposicao`) faz `IF EXISTS` antes de aceitar o INSERT. Em isolamento `READ COMMITTED` (default), dois `INSERT`s concorrentes em sessões diferentes podem ambos passar o IF antes de qualquer COMMIT — race condition: ambas as reservas ficam gravadas.

Solução adotada: `sp_getapplock` nos SPs `sp_criar_reserva_sala` e `sp_criar_reserva_posto`, com resource string `'reserva_recurso_<id>_<data>'`. O lock é exclusive e libertado no fim da transacção (`@LockOwner = 'Transaction'`). Sessões

concorrentes para o mesmo recurso/data serializam-se; sessões para recursos diferentes não bloqueiam.

Alternativa considerada: `ALLOW_SNAPSHOT_ISOLATION ON` + `SET TRANSACTION ISOLATION LEVEL SERIALIZABLE`. Rejeitada porque sobrecarrega todas as queries (não só as de reserva) e degrada throughput.

Ver `Tests/test_concorrencia.sql` para reprodução manual em duas sessões SSMS.

7. Auditoria — Temporal Tables

`adesao`, `reserva` e `pagamento` são `SYSTEM_VERSIONED`. Cada `UPDATE` / `DELETE` escreve a versão anterior em `<tabela>_history` com `valid_from` / `valid_to` automáticos. Permite:

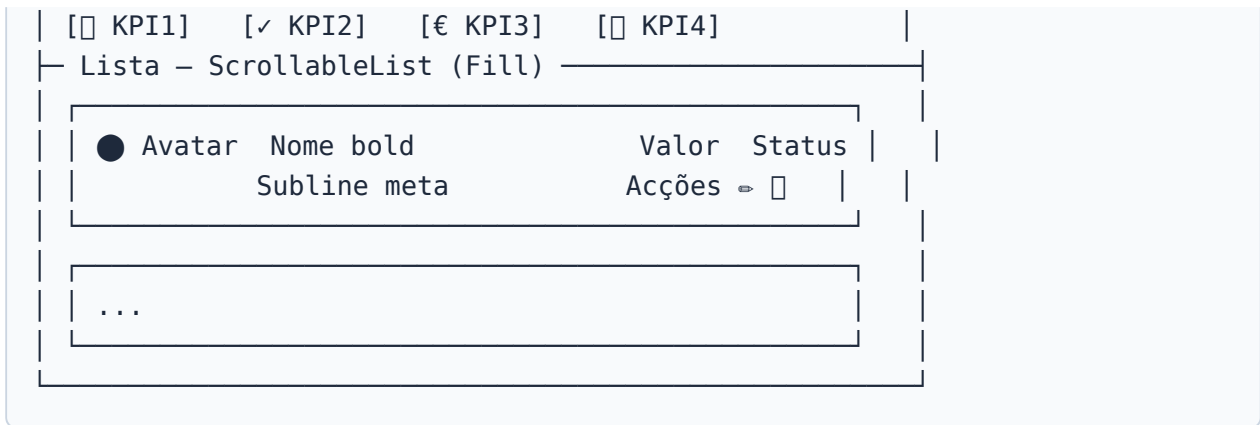
- Reconstituir o estado da BD em qualquer momento: `SELECT * FROM reserva FOR SYSTEM_TIME AS OF '2026-05-01';`
- Auditar quem alterou o quê (cruzando com logs de `sys.dm_exec_sessions`).
- Suportar relatórios de evolução (revenue mensal histórico imutável).

Pré-requisito: nenhuma tabela `SYSTEM_VERSIONED` pode ter trigger `INSTEAD OF`. O trigger original `trg_adesao_preco_snapshot` foi removido e a lógica de snapshot do preço migrou para `sp_criar_adesao`.

8. Performance e Planos de Execução

Índices criados (ver `Indexes.sql`):

Índice	Coluna(s)	Query que otimiza
<code>idx_reserva_recurso</code>	(recurso_id, data_reserva, hora_inicio, hora_fim)	Trigger T1: scan de sobreposições
<code>idx_pagamento_adesao</code> (filtered)	adesao_id WHERE NOT NULL	Trigger T6: validação valor
<code>idx_pagamento_reserva</code> (filtered)	reserva_id WHERE NOT NULL	Trigger T6: validação valor
<code>idx_adesao_cliente</code>	(cliente_id, estado)	Trigger T4: adesão ativa única
<code>idx_reserva_cliente</code>	(cliente_id, estado)	Listagem "as minhas reservas"
<code>idx_pagamento_cliente</code>	(cliente_id, estado)	<code>fn_receita_cliente</code> , <code>vw_top_clientes</code>



Decisão arquitectural: o **outer** do conteúdo de cada tab é um `Panel(PageBg)`, não um `ModernCard(CardBg)`, para que os cards interiores tenham contraste visual contra o background mais escuro do parent.

Componentes reutilizáveis novos

Componente	Responsabilidade
<code>TabButton</code>	Tab com underline accent + <code>AutoSize</code> por texto
<code>StatusPill</code>	Pill <code>Filled</code> ou <code>Dot</code> ; helper <code>MeasureDotWidth</code> (via <code>Graphics.MeasureString</code> , mais preciso que <code>TextRenderer.MeasureText</code> para fonts bold pequenos)
<code>ModernCombo</code>	Wrapper visual de <code>ComboBox</code> com bg <code>FieldBg</code> rounded
<code>ModernSelect</code>	Dropdown custom com popup <code>Form</code> borderless + <code>FocusablePanel</code> pintado à mão (substitui <code>ComboBox</code> cuja chrome branca não respeitava o tema dark)
<code>ModernDateField</code> + <code>ModernCalendar</code>	Campo data + popup calendar custom em pt-PT (substitui <code>DateTimePicker</code> / <code>MonthCalendar</code> nativos Windows-style)
<code>SegmentedControl</code>	Picker `[A
<code>ScrollableList</code>	<code>Panel</code> scrollable com scrollbar dark + thumb arrastável (substitui o <code>AutoScroll</code> nativo que mostrava scrollbar branca)
<code>ToggleChip</code>	Chip on/off para filtros booleanos

Bugs notáveis resolvidos durante o redesign

- **Vírgula clipada nos preços** (`200,00 €` parecia `200.00 €`). Labels com `Height=22` e `Font 12pt bold` cortavam o descender da vírgula. Padronizado para `Height ≥ 28` em todos os preços.
- **Bars empilhadas no mesmo X** no chart de Receita Mensal. `Series.IsXValueIndexed = true` força index incremental por ponto em vez de o framework tentar parsear o X string

como número (e falhar → tudo a `X=0`).

- **Bordas brancas do `ComboBox` nativo** que não respeitavam o tema dark — resolvido criando o `ModernSelect` from scratch.
- **Pills com último char cortado** (`"Terminada"` → `"Terminad"`).
`TextRenderer.MeasureText` subestima 1–3 px o glyph real vs `Graphics.MeasureString` (engine GDI+). Standardizado via `StatusPill.MeasureDotWidth` que usa o último + buffer.
- **Win32 handle exhaustion** com seed expandido (~290 reservas × ~15 controls por card = 4350+ handles num único UC). Limit do processo ≈ 10000. Resolvido com `MaxRender = 80` em cada UC + label "+ X mais antigos não mostrados", e `TOP 200` nas queries.
- **Popup do detalhe de notificação fechava-se sozinho** ao abrir — `Form.Deactivate` disparava durante o `Show()` por race com foco. Substituído por `IMessageFilter` que detecta `WM_LBUTTONDOWN` fora de `Form.Bounds`.

Métricas

- 13 commits dedicados a esta fase (`feature/*-redesign` branches, merge `--no-ff` a cada UC concluído).
- ~3500 linhas novas (componentes + redesigns) contra ~1800 removidas (DataGridViews + toolbar antiga).
- 9 novos ficheiros de componentes (`TabButton.cs` , `StatusPill.cs` , `ModernCombo.cs` , `ModernSelect.cs` , `ModernDateField.cs` , `ModernCalendar.cs` , `SegmentedControl.cs` , `ScrollableList.cs` , `ToggleChip.cs`).

Seed expandido para popular charts

`SQL_DML.sql` ganhou um bloco final (preserva o seed base original) com:

- 15 clientes adicionais (total 20)
- 25 adesões históricas via `INSERT` literal (20 `Terminada` + 5 `Ativa/Pendente`)
- ~72 reservas de sala via `WHILE` loop (1 quarta-feira/semana × 72 semanas), com rotação determinística de clientes/recursos e slots horários para evitar T1 (sobreposição)
- ~9 day passes de posto Flex (Diogo/Eva alternados, ~bimestrais — evita T11)
- Pagamentos auto-gerados via `INSERT ... SELECT` sobre `adesao` e `reserva` (snapshot=valor para passar T6)

Cobertura: 18 meses (Jan/2025 → Mai/2026), suficiente para os gráficos mostrarem tendências mensais sem sobrecarregar a UI.

Ordem de execução dos scripts SQL

A ordem é crítica porque alguns SPs dependem de outros. Documentada em

`SQL_Scripts/README.md` :

SQL_DDL → User_defined_functions → Triggers → Views → Stored_procedures
→ Temporal_tables → Indexes → SQL_DML

11. Conclusão

O sistema cobre o domínio com 9 tabelas core + 3 de extensão (política de cancelamento, notificação, lista de espera), com regras enforçadas a 3 níveis: constraints SQL (cardinalidade/domínio), triggers (regras transversais), SPs (concorrência + lógica composta). A aplicação invoca a BD exclusivamente via SPs parametrizadas — resistente a SQL injection por construção — e a auditoria via temporal tables dá-nos histórico para sempre sem trigger adicional.

Limitações conhecidas:

- Notificações ficam só na tabela; envio efetivo de email exigiria SQL Agent + Database Mail configurados fora do scope da disciplina.
- O cálculo de reembolso assume que a política está seedada com tiers cobrindo 0h em diante; falta UI para gerir tiers.
- O sistema não suporta multi-tenancy (uma só CoworkingDB serve um operador).

Próximos passos naturais: dashboard de KPIs em SSRS, UI de gestão de políticas, integração com gateway de pagamento real.

Anexo A. Rastreabilidade Requisito → Implementação

Mapa de cobertura entre o **Requisitos.md** (APF-E) e o schema/SQL/app implementado. Para cada item, lista-se a implementação concreta — útil para auditoria do professor e como suporte à apresentação oral.

Convenções: **T1..T16** = triggers; **sp_*** = stored procedures; **vw_*** = views; **fn_*** = funções; **UcX** = UserControl C#; CHECK/UNIQUE/FK = constraints declarativas.

A.1 Requisitos Funcionais

RF	Descrição	Implementação
RF1	Registrar clientes	Tabela cliente ; SP sp_registar_cliente ; UcClientes (CRUD)
RF2	Dados de identificação e	Colunas cliente.nome, nif, email, telefone, data_registro ; UNIQUE em nif e email ; CHECK nif NOT

	contacto	LIKE '%[^0-9]%'
RF3	Registar planos	Tabela <code>plano</code> ; UNIQUE <code>nome_plano</code> ; CHECK <code>tipo_plano</code> ∈ {Flex,Fixo,Privado} ; <code>UcPlanos</code>
RF4	Associar clientes a planos via adesões	Tabela <code>adesao</code> (FK <code>cliente_id</code> + FK <code>plano_id</code>) ; SP <code>sp_criar_adesao</code> ; <code>UcAdesoes</code>
RF5	Manter histórico de adesões	<code>adesao.estado</code> ∈ {Pendente,Ativa,Suspensa,Cancelada,Terminada} ; SYSTEM_VERSIONING → tabela <code>adesao_history</code> ; view <code>vw_clientes_com_adesao_ativa</code>
RF6	Registar espaços físicos	Tabela <code>espaco</code> ; UNIQUE <code>nome</code> ; CHECK <code>hora_fecho</code> > <code>hora_abertura</code> ; <code>UcEspacos</code>
RF7	Salas em cada espaço	Tabela <code>sala</code> (FK <code>espaco_id</code>) ; UNIQUE (<code>espaco_id</code> , <code>nome</code>) ; <code>UcEspacos</code> (tab Salas)
RF8	Postos em cada espaço	Tabela <code>posto</code> (FK <code>espaco_id</code>) ; UNIQUE (<code>espaco_id</code> , <code>codigo</code>) ; <code>UcEspacos</code> (tab Postos)
RF9	Reservar salas	<code>sp_criar_reserva_sala</code> (com <code>sp_getapplock</code>) ; T2/T3/T12 validam horário/capacidade/coerência ; <code>UcReservas</code>
RF10	Reservar postos	<code>sp_criar_reserva_posto</code> (com <code>sp_getapplock</code>) ; T11/T12 validam compatibilidade com adesão ; <code>UcReservas</code>
RF11	Histórico de reservas de um cliente	UDF <code>fn_reservas_cliente_periodo(@cli, @ini, @fim)</code> ; view <code>vw_reservas_historico</code> (FOR SYSTEM_TIME ALL) ; <code>UcRelatorios</code>
RF12	Consultar ocupação de salas/postos	View <code>vw_ocupacao_recurso</code> ; UDF <code>fn_taxa_ocupacao_espaco(@esp, @data)</code> ; UDF <code>fn_recurso_disponivel(...)</code> ; <code>UcDashboard</code> , <code>UcEstatisticas</code>
RF13	Registar pagamentos	Tabela <code>pagamento</code> ; SP <code>sp_registar_pagamento</code> (com snapshot) ; <code>UcPagamentos</code>
RF14	Histórico de pagamentos por cliente	UDF <code>fn_receita_cliente(@cli)</code> ; view <code>vw_top_clientes_receita</code> ; SYSTEM_VERSIONING → <code>pagamento_history</code> ; <code>UcRelatorios</code>
RF15	Controlar estado de reservas e adesões	Colunas <code>estado</code> em ambas (CHECK enum) ; SPs <code>sp_cancelar_reserva</code> , <code>sp_cancelar_reserva_com_reembolso</code> , <code>sp_cancelar_adesao</code> ; trigger T5 calcula <code>data_fim</code>

A.2 Requisitos Não Funcionais

RNF	Descrição	Implementação
RNF1	Integridade e consistência	FKs em todas as relações; 16 triggers; SPs com <code>SET XACT_ABORT ON</code> + <code>sp_getapplock</code> para concorrência; CHECK em domínios de enum
RNF2	Unicidade NIF e email	UNIQUE em <code>cliente.nif</code> e <code>cliente.email</code> (constraints declarativas — falham com erro 2627/2601)
RNF3	Consulta fácil de disponibilidade	UDF <code>fn_recurso_disponivel</code> ; view <code>vw_ocupacao_recurso</code> ; index em <code>(recurso_id, data_reserva)</code>
RNF4	Crescimento de dados	<code>Indexes.sql</code> cobre FKs e colunas de busca frequente; views agregadas (<code>vw_receita_mensal</code> , <code>vw_receita_por_plano</code>) suportam paginação
RNF5	Clareza do modelo	DER + ER actualizados (<code>presentation/DER.md</code> , <code>presentation/EsquemaRelacional.md</code>); §3 do relatório justifica cada decisão

A.3 Regras de Negócio

RN	Descrição	Implementação
RN1	Cliente pode ter 0..N adesões	Cardinalidade FK em <code>adesao.cliente_id</code> (1:N). Desvio: T4 (<code>trg_adesao_ativa_unica</code>) restringe a uma única adesão Ativa simultânea — decisão consciente discutida em §3.2 (evita conflitos de faturação)
RN2	Adesão pertence a 1 cliente e 1 plano	FK NOT NULL em <code>adesao.cliente_id</code> e <code>adesao.plano_id</code>
RN3	Plano associado a vários clientes ao longo do tempo	Cardinalidade N:1 em <code>adesao</code> → <code>plano</code> ; view <code>vw_receita_por_plano</code> agrega histórico
RN4	Espaço inclui várias salas	FK <code>sala.espaco_id</code> (N:1) com <code>ON DELETE NO ACTION</code> (não permite apagar espaço com salas)
RN5	Espaço disponibiliza vários postos	Analogamente, <code>posto.espaco_id</code> (N:1) com <code>ON DELETE NO ACTION</code>
RN6	Cada sala pertence a 1 espaço	FK NOT NULL <code>sala.espaco_id</code>

RN7	Cada posto pertence a 1 espaço	FK NOT NULL <code>posto.espaco_id</code>
RN8	Cliente pode reservar várias salas	Cardinalidade FK <code>reserva.cliente_id</code> (N:1)
RN9	Cliente pode reservar vários postos	Mesma — tabela <code>reserva</code> é unificada (correção #2 do prof.)
RN10	Reserva de sala refere-se a 1 sala	FK <code>reserva.recurso_id</code> ; T12 (<code>trg_reserva_horas_coerentes</code>) exige <code>hora_inicio/fim</code> NOT NULL quando recurso é sala
RN11	Reserva de posto refere-se a 1 posto	FK <code>reserva.recurso_id</code> ; T12 exige horas NULL; T11 evita colisão com adesão Flex/Fixo/Privado activa
RN12	Sala sem reservas sobrepostas	T1 (<code>trg_reserva_sem_sobreposicao</code>) + <code>sp_criar_reserva_sala</code> com <code>sp_getapplock</code> por (<code>recurso_id, data</code>) para evitar race condition
RN13	Posto sem reservas sobrepostas	Mesmo T1 + <code>sp_criar_reserva_posto</code> (dia inteiro: compara só data, não horas)
RN14	Cliente pode efectuar vários pagamentos	Cardinalidade FK <code>pagamento.cliente_id</code> (N:1)
RN15	Cada pagamento associado a 1 cliente	FK NOT NULL <code>pagamento.cliente_id</code> ; T8 (<code>trg_pagamento_cliente_consistente</code>) valida que <code>cliente_id</code> é o titular do serviço pago

A.4 Correções do Professor

Correção	Implementação
#1 — "Qual o preço?" do Pagamento	Coluna <code>pagamento.preco_servico_snapshot</code> DECIMAL(10,2) NOT NULL ; CHECK <code>ck_pagamento_valor_snapshot</code> (<code>valor = preco_servico_snapshot</code>) ; trigger T6 (<code>trg_pagamento_valor_correto</code>) valida que <code>preco_servico_snapshot</code> coincide com <code>reserva.valor / adesao.preco_acordado</code> ; SP <code>sp_registar_pagamento</code> faz o snapshot a partir do serviço. Justificação em §3.2
#2 — Reserva como	Tabela única <code>reserva</code> (em vez de <code>reserva_sala</code> + <code>reserva_posto</code> separadas) com FK para <code>recurso</code> (supertype de <code>sala</code> / <code>posto</code>). M:N entre <code>cliente</code> e <code>recurso</code> materializada com atributos próprios (<code>data_reserva</code> ,

entidade associativa	<code>hora_*</code> , <code>valor</code> , <code>estado</code> , <code>num_participantes</code> , <code>notas</code>). T12 enforça as semânticas diferentes consoante o tipo de recurso
-----------------------------	--

A.5 Features adicionais (não pedidas pelos requisitos)

Feature	Motivação	Implementação
Política de cancelamento com reembolso	Reflectir prática real de coworking	Tabela <code>politica_cancelamento</code> (tiers por antecedência); SP <code>sp_cancelar_reserva_com_reembolso</code> aplica a tier de maior <code>horas_minimas ≤ antecedência</code>
Notificações	Feedback ao cliente sobre eventos relevantes	Tabela <code>notificacao</code> ; triggers T13/T14/T15 emitem automaticamente (criar/cancelar reserva, confirmar pagamento); view <code>vw_notificacoes_por_ler</code> ; <code>UcNotificacoes</code>
Lista de espera	Recurso indisponível não deve perder o cliente	Tabela <code>lista_espera</code> ; SPs <code>sp_adicionar_lista_espera</code> , <code>sp_promover_lista_espera</code> (cria reserva + actualiza estado + notifica)
Reservas recorrentes	Use case comum em coworking	SP <code>sp_criar_reserva_recorrente</code> (cria N reservas por dia da semana, com try/catch individual para reportar falhas)
Auditoria temporal	Recuperar histórico sem schema adicional	<code>SYSTEM_VERSIONING = ON</code> em <code>adesao</code> , <code>reserva</code> , <code>pagamento</code> → tabelas <code>_history</code> auto-mantidas; view <code>vw_reservas_historico</code> (FOR SYSTEM_TIME ALL)
Indicador de ligação à BD	Feedback visual ao utilizador se a BD está acessível	Status bar do <code>FormMain</code> com dot verde/vermelho + texto LIGADO/SEM LIGAÇÃO; <code>Database.IsAvailable()</code> chamado on-load + a cada 30s (assíncrono)
Tab "Mapa" em Reservas	Visualizar ocupação por hora num dia (gantt simples)	<code>MapaReservasView</code> (Panel custom-painted) com timeline 08h-20h, linhas=salas agrupadas por espaço, células coloridas pelo estado da reserva; click abre detail dialog
Recibo PDF de pagamento	Cliente/staff precisa de documento de prova	Pacote <code>PdfSharp 6.1.1</code> (MIT); <code>ReciboPdf.cs</code> gera A4 com header indigo, cliente, serviço, snapshot de preço, total destacado; botão "↓ Recibo PDF" no detalhe do pagamento